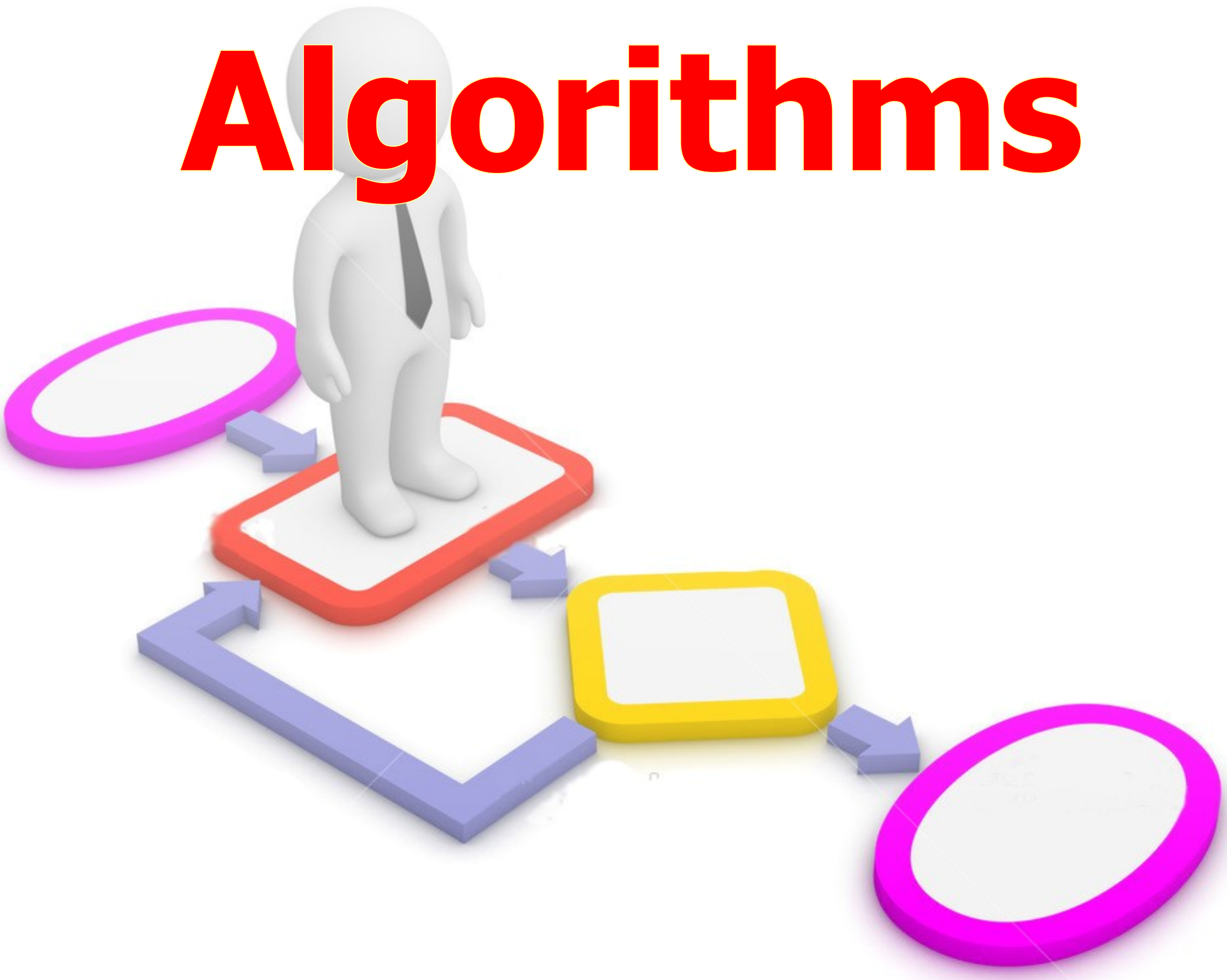
Algorithms



Published By:

Physics Wallah

ISBN: 978-93-94342-39-2

Mobile App: Physics Wallah (Available on Play Store)

Website: www.pw.live

Email: support@pw.live

Rights

Disclaimer

A team of PW experts and faculties with an understanding of the subject has worked hard for the books.

While the author and publisher have used their best efforts in preparing these books. The content has been checked for accuracy. As the book is intended for educational purposes, the author shall not be responsible for any errors contained in the book.

The publication is designed to provide accurate and authoritative information with regard to the subject matter covered.

(This Module shall only be Used for Educational Purpose.)

Algorithm

INDEX

1. Asymptotic Notation .... 6.1 – 6.18
2. Divide and Conquer .... 6.19 – 6.25
3. Greedy Technique .... 6.26 – 6.29
4. Dynamic Programming .... 6.30 – 6.34

1 ASYMPTOTIC NOTATION

1.1 Introduction of Course

Algorithms

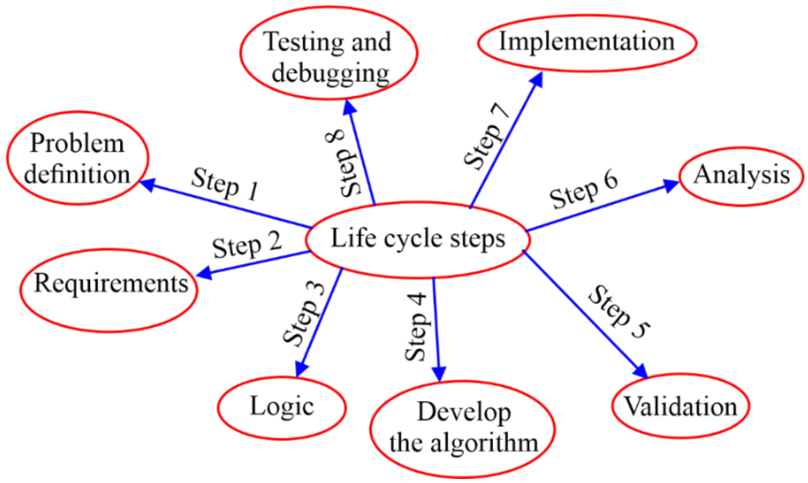
- **Analysis of Algorithms**
  - Algorithm concept and life cycle
  - Needs for analysis
  - Methodology of analysis
  - Types of analysis
  - Asymptotic notation
  - Framework for analysis of recursive and non recursive Algorithms
  - Analysing loop
  - Space complexities
- **Design Strategies**
  - Divide and Conquer
  - Greedy method
  - Dynamic programming
- **Graph Algorithms**
  - Graph Algorithms
  - Heap Algorithms
  - Hashing
  - Sets
  - Sorting Algorithms

1.2 Algorithm Concept and Life Cycle Steps

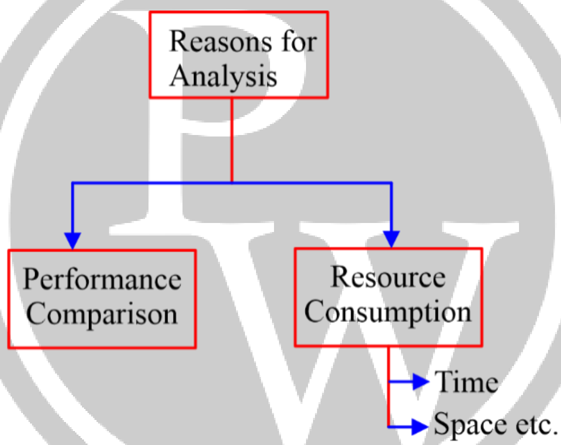
1.2.1 Algorithm

- An Algorithm consists finite number of steps to solve any problem.
- Every step involves some operations and each operation must be definite and effective.

1.2.2 Life Cycle Steps



1.3 Needs of Analysis



In performance comparison comparing different algorithms for optimal solution.

1.3.1 Time Complexity

Time complexity of an algorithm quantifies the amount of time taken by an algorithm to run as a function of the input size.

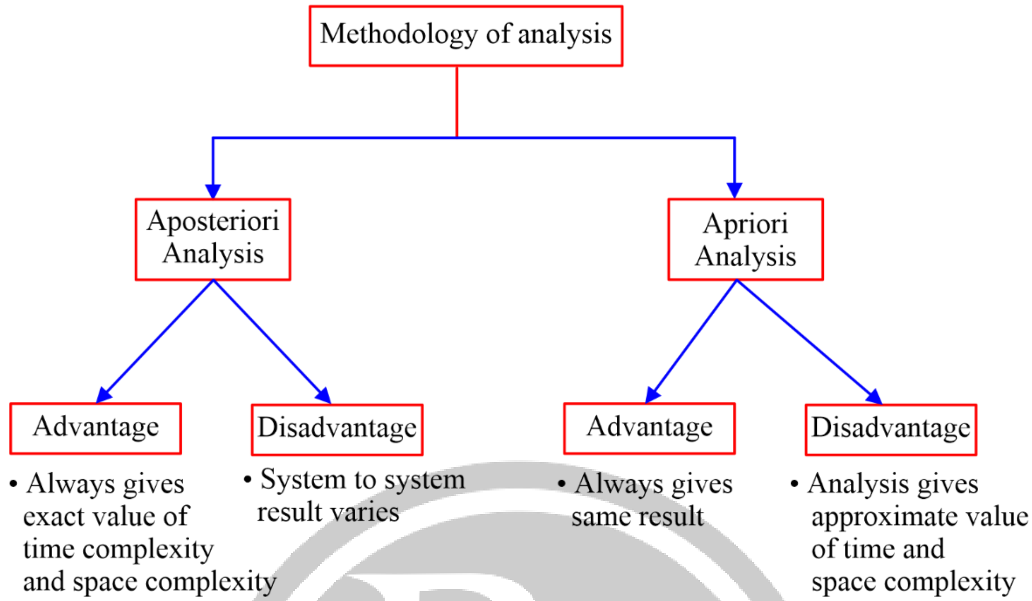
1.3.2 Space Complexity

Space complexity of an algorithm quantifies the amount of space or memory taken by an algorithm to run as a function of input size.

Note:

To find the time complexity of an algorithm, find the loops and also consider larger loops.
Space complexity is dependent on two things input size and some extra space (stack space link, space list etc).

1.4 Methodology of Analysis



1.5 Types of Analysis

Worst Case

The input class for which the algorithm does maximum work and hence, take maximum time.

Best Case

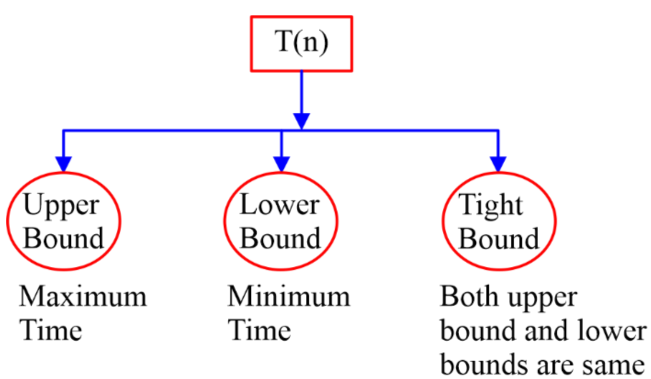
The input class for which the algorithm does minimum work hence, take minimum time.

Average Case

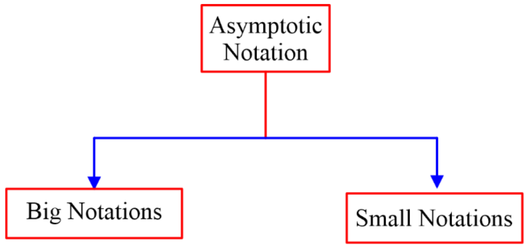
Average case can be calculated form best case to worst case.

1.6 Asymptotic Notations

Suppose, T(n) be a function of time for any algorithm.



1.7 Types of Asymptotic Notations



- Big oh (O): Upper bound
- Big omega (Ω): Lower bound
- Theta (θ): Tight bound

- Small oh (o): Proper upper bound
- Small omega (ω): Proper lower bound

1.7.1 Big O – Notation

Two Functions $f(n), g(n)$

$f(n) = O(g(n))$

When the growth of $g(n)$ is same or higher than $f(n)$ like $a \le b$

Example:

$f(n) = 3n + 10, g(n) = n^2 + 2n + 5$

$f(n) = O(g(n))$

1.7.2 Ω - Notation

$f(n) = \Omega(g(n))$

$\therefore f(n) \ge C \cdot g(n) \qquad (a \ge b)$

Example: $3^n = \Omega(2^n)$

1.7.3 θ - Notation

If $f(n) \le g(n)$

And

$f(n) \ge g(n)$

$f(n) = g(n)$

$\therefore \quad f(n) = \theta(g(n))$

Example:

$f(n) = 2n^2, g(n) = n+10$

$f(n) > g(n)$ here

so, $f(n) = \Omega(g(n))$ or $g(n) = O(f(n))$

1.7.4. Properties with respect to asymptotic notations

	Reflexive	Symmetric	Transitive	Transpose symmetric
Big oh (O)	✓	×	✓	✓
Big omega (Ω)	✓	×	✓	✓
Theta (θ)	✓	✓	✓	×
Small oh (o)	×	×	✓	✓
Small omega (ω)	×	×	✓	✓

Example 1. Consider the following function

$$f(n) = \sum_{p=1}^{n} p^3 = q$$

Which of the following is/are true for 'q'

(a) $\theta(n^4)$ (b) $\theta(n^5)$ (c) $O(n^5)$ (d) $\Omega(n^3)$

Solution: (a, c, d)

$$f(n) = \sum_{P=1}^{n} P^3$$

$$= 1^3 + 2^3 + 3^3 + 4^3 \ldots\ldots\ldots\ldots\ldots + n^3$$

$$= \left(n\left(\frac{n+1}{2}\right)\right)^2$$

$$= O(n^4) \text{ or } \Omega\,(n^4)$$

$$= \theta\,(n^4)$$

Example 2. Consider the following functions:

$$f(n) = \sum_{P=1}^{n} P^{1/2} = q$$

Find the value of q in terms of asymptotic notation.

Solution: $f(n) = \sum_{P=1}^{n} P^{1/2}$

$$= 1 + (2)^{1/2} + (3)^{1/2} + \ldots\ldots$$

$$= \frac{2}{3}\left[n^{3/2} - 1\right]$$

$$= \frac{2}{3}n^{3/2} - \frac{2}{3}$$

$$= O(n^{1.5})$$

$$= O(n\sqrt{n})$$

Example 3. Arrange the following functions in increasing order.

$$f_1 = n\log n,\ f_2 = \sqrt{n},\ f_3 = 2^n,\ f_4 = 3^n,\ f_5 = n!,\ f_6 = n^n,\ f_7 = \sqrt{\log n},\ f_8 = 100n\log n$$

$\rightarrow f_7 < f_2 < fl = f_8 < f_3 < f_4 < f_5 < f_6$

Example 4. Arrange the following functions in increasing order.

$$f_1 = 10,\ f_2 = \sqrt{n},\ f_3 = \log\log n,\ f_4 = (\log n)^2,\ f_5 = n^2$$

$$f_6 = n\log n,\ f_7 = n!,\ f_8 = 2^n,\ f_9 = n^n,\ f_{10} = n^2\log n$$

$$\rightarrow\ f_1 < f_3 < f_4 < f_2 < f_6 < f_5 < f_{10} < f_8 < f_7 < f_9$$

Example 5. Arrange the following functions in increasing order.

$f_1 = \log\log n \qquad f_9 = n\log\log n$

$f_2 = \log n \qquad f_{10} = n^2\log n$

$f_3 = (\log n)^2 \qquad f_{11} = n^3$

$f_4 = \sqrt{\log n} \qquad f_{12} = 2^n$

$f_5 = n^{1/10} \qquad f_{13} = e^n$

$f_6 = n \qquad f_{14} = n!$

$f_7 = n^2 \qquad f_{15} = n^n$

$f_8 = n\log n \qquad f_{16} = n^{3/2}$

$$f_1 < f_4 < f_2 < f_3 < f_5 < f_6 < f_9 < f_8 < f_{16} < f_7 < f_{10} < f_{11} < f_{12} < f_{13} < f_{14} < f_{15}$$

$$a^{\log_b c} \Leftrightarrow c^{\log_b a}$$

$$\therefore 2^{\log_2 n} \Leftrightarrow n^{\log_2 2} = n$$

Example 6. Arrange the following functions in increasing order.

$$f_1 = n!,\ f_2 = n^n$$

$$f_1 = n\times(n-1)(n-2)\times\ldots\times 3\times 2\times 1$$

$$f_2 = n\times n\times n\times n\times\ldots\times n\times n\times n$$

$$f_2 > f_1$$

$$\therefore f_1 = O(f_2)$$

$$\boxed{2^n < 3^n < 4^n < n! < n^n}$$

Question.

Which of following is TRUE?

(1) $2^{\log_2 n} = O(n^2)$ TRUE

(2) $n^2 \cdot 2^{3\log_2 n} = O(n^5)$ TRUE

(3) $2^n = O(2^{2n})$ TRUE

(4) $\log n = O(\log\log n)$ FALSE

(5) $\log\log n = O(n\log n)$ TRUE

Solution:

(1) $2^{\log_2 n} = O(n^2)$

$$= n^{\log_2 2}$$

$$= n$$